โครงการเครือข่ายสื่อสารระดับจักรวาลที่อ้างอิงด้วยเหตุ-ผลเป็นศูนย์กลาง

รายงานชุดเอกสารประกอบโครงการ - หลักสูตรวิทยาการคอมพิวเตอร์

วิชา CP352005 เครือข่ายคอมพิวเตอร์ (Computer Networks)

ระยะเวลาดำเนินโครงการ: 4 สัปดาห์

ทีมพัฒนา: 5 บทบาท

ไฟล์ที่ 1: architecture_spec.md

Universal Causality-based Communication Network (UCCN) - สถาปัตยกรรมเครือข่ายเวอร์ชัน 1.0

การควบคุมเอกสาร

เวอร์ชัน	วันที่	ผู้จัดทำ	บทบาท	การเปลี่ยนแปลง
v1.0	22/2/2026	นายวงศธร ธน.ยอด	สถาปนิกระบบ	จัดทำสถาปัตยกรรมเบื้องต้น

บทบาททีมพัฒนา

บทบาท	ชื่อ	ความรับผิดชอบหลัก
สถาปนิกระบบ (Architect)	นายวงศธร ธน.ยอด	ออกแบบระบบโดยรวม, กำหนดเลเยอร์, สร้างข้อตกลงระหว่างส่วนประกอบ
วิศวกรเครือข่าย (Engineer)	นายพัชรพล กองแก้ว	พัฒนาโพรโทคอล, พัฒนาจำลองสถานการณ์, เขียนโค้ดหลัก
ผู้เชี่ยวชาญเฉพาะด้าน (Specialist)	นายกฤษฎา นามมนต์เทียน	วิจัยความสัมพันธ์เชิงเหตุผล, กฎ Paradox, องค์ความรู้เฉพาะทาง
ผู้ดูแลระบบและเครื่องมือ (DevOps)	นายศุภกิตติ์ ฟันเฟือย	จัดการสภาพแวดล้อม, CI/CD, บูรณาการ, ดูแลเอกสาร

ผู้ทดสอบ/ประกันคุณภาพ (Tester/QA)	นายคมชาญ น้อยเนียม	วางแผนทดสอบ, ออกแบบกรณีทดสอบ, ตรวจสอบคุณภาพ

ส่วนที่ 1: บทสรุปผู้บริหาร

1.1 วิสัยทัศน์ของโครงการ

UCCN (Universal Causality-based Communication Network) เป็นสถาปัตยกรรมเครือข่ายเชิงทฤษฎีที่นำแนวคิดการอ้างอิงด้วย "เหตุ-ผล" (Causality-based Referencing) มาใช้แทนการอ้างอิงด้วยตำแหน่งทางกายภาพแบบเครือข่ายดั้งเดิม โครงการนี้ประยุกต์แนวคิดจาก 5 แนวคิดย่อยของทีมเข้าด้วยกัน:

1. แนวคิดหลัก: Causality-based Network (โดยนายพัชรพล) - เครือข่ายที่อ้างอิงด้วยเหตุ-ผล ไม่ใช่ที่อยู่
2. สื่อกลางในสภาพแวดล้อมสุดขั้ว: Neutrino Communication (โดยนายศุภกิตติ์)
3. สื่อกลางทางเลือก: Gravitational Wave Communication (โดยนายกฤษฎา)
4. กลไกความน่าเชื่อถือ: Delay-Tolerant Networking (โดยนายคมชาญ)
5. ชั้นพื้นฐาน: Photon-based Communication (โดยนายวงศธร)

โครงการนี้สาธิตความเข้าใจในการออกแบบเครือข่ายแบบมีเลเยอร์ โพรโทคอลการค้นหาเส้นทาง รูปแบบการระบุแอดเดรส และการรักษาความปลอดภัยเครือข่าย (ผ่านการป้องกัน Paradox) ภายใต้ข้อจำกัดของโครงการระดับปริญญาตรี

1.2 วัตถุประสงค์ทางการศึกษา

- ประยุกต์แนวคิด OSI/TCP-IP กับสถานการณ์แปลกใหม่
- ออกแบบและระบุรายละเอียดโพรโทคอลเครือข่าย
- พัฒนาอัลกอริทึมการค้นหาเส้นทางด้วยเมตริกเฉพาะทาง
- สร้างการจำลองสถานการณ์เครือข่าย
- ฝึกปฏิบัติเขียนเอกสารสถาปัตยกรรม
- เข้าใจความปลอดภัยเครือข่ายผ่านการเปรียบเทียบกับการป้องกัน Paradox

1.3 ขอบเขตและข้อจำกัดของโครงการ

ด้าน	**อยู่ในขอบเขต**	**นอกขอบเขต**

สถาปัตยกรรม	การออกแบบแบบมีเลเยอร์, โพรโทคอล, รูปแบบแอดเดรส	การใช้งานจริงทางกายภาพ
การจำลองสถานการณ์	การจำลองเครือข่ายด้วย Python	การใช้งานฮาร์ดแวร์/ควอนตัมจริง
มิติเหตุ-ผล	การแทนด้วยโครงสร้างข้อมูล	ความเป็นจริงทางกายภาพ
โพรโทคอล	CAAP, TRP, PPP, TSP	โพรโทคอลระดับพร้อมใช้งานจริง
การทดสอบ	การทดสอบหน่วย, การทดสอบบูรณาการ	การใช้งานในโลกจริง

ส่วนที่ 2: การทบทวนสถาปัตยกรรม

2.1 ภาพรวมสถาปัตยกรรม

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          UCCN Protocol Stack                                 │
├──────────────────────────────────────────────────────────────────────────────┤
│ Layer 7: Application Layer                                                   │
│ [Cosmic Knowledge Sync, Interstellar Messaging,                              │
│  Civilization Archive, Event Horizon Monitor]                                │
├──────────────────────────────────────────────────────────────────────────────┤
│ Layer 6: Presentation Layer                                                  │
│ [Causal Serialization, Multi-Modal Data Encoding,                            │
│  Timeline Adaptation, Reality Reconciliation]                                │
├──────────────────────────────────────────────────────────────────────────────┤
│ Layer 5: Session Layer                                                       │
│ [Timeline Synchronization Protocol (TSP),                                    │
│  Causal Session Management, Interstellar Handshake]                          │
├──────────────────────────────────────────────────────────────────────────────┤
│ Layer 4: Transport Layer                                                     │
│ [Paradox Prevention Protocol (PPP),                                          │
│  Causal Reliability Control, Multi-Path Temporal Transport]                  │
├──────────────────────────────────────────────────────────────────────────────┤
│ Layer 3: Network Layer                                                       │
│ [Temporal Routing Protocol (TRP),                                            │
│  Causal Path Discovery, Interstellar Forwarding]                             │
├──────────────────────────────────────────────────────────────────────────────┤
│ Layer 2: Data Link Layer                                                     │
│ [Causal-Anchor Addressing Protocol (CAAP),                                   │
│  Multi-Modal Link Adaptation, Frame Synchronization]                         │
├──────────────────────────────────────────────────────────────────────────────┤
│ Layer 1: Physical Layer (Simulated Multi-Modal)                              │
│  ┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐             │
│  │ Photon           │ │ Neutrino         │ │ Gravitational    │ │ DTN Store-  │ │
│  │ Channel          │ │ Channel          │ │ Wave Channel     │ │ &-Forward   │ │
│  └──────────────────┘ └──────────────────┘ └──────────────────┘             │
└──────────────────────────────────────────────────────────────────────────────┘
```

2.2 การทบทวนสถาปัตยกรรมรายเลเยอร์

2.2.1 เลเยอร์ที่ 1: Physical Layer (Multi-Modal Simulation)

สถานะการตรวจสอบ: อนุมัติพร้อมข้อสังเกต

มิติ	การประเมิน	ข้อสังเกต
ความสมบูรณ์	สูง	กำหนดอินเทอร์เฟซชัดเจน รองรับ 4 ช่องทาง
ความเป็นไปได้	สูง	จำลองด้วย Python ได้
การบูรณาการ	ปานกลาง	ต้องออกแบบการสลับช่องทางอัตโนมัติ
ความคิดสร้างสรรค์	สูง	ผสาน 4 สื่อกลางในเลเยอร์เดียว

คำจำกัดความอินเทอร์เฟซ

```
class PhysicalLayer:
    def __init__(self):
    self.channels = {
    'photon': PhotonChannel(),
    'neutrino': NeutrinoChannel(),
    'gravitational': GravitationalWaveChannel(),
    'dtn_store': DTNStoreChannel()
    }
    def transmit(self, packet, source_ca, dest_ca, preferred_channel=None):
    ส่งแพ็กเกตผ่านช่องทางที่เหมาะสม
    - เลือกช่องทางอัตโนมัติตามสภาพแวดล้อม
    - รองรับการระบุช่องทางที่ต้องการ
    def get_channel_metrics(self, channel_type, source_ca, dest_ca):
    คืนค่าเมตริกของช่องทาง:
    - latency: ความหน่วง
    - bandwidth: แบนด์วิธ
    - interference: สัญญาณรบกวน
    - Penetrating ability: ความสามารถในการทะลุผ่านสสาร
```

คุณสมบัติช่องทาง:

ช่องทาง	ความเร็ว	ทะลุสสาร	เหมาะกับ
Photon	c	ต่ำ	สภาพปกติ, ต้องการแบนด์วิธสูง
Neutrino	≈c	สูงมาก	ใจกลางดาวฤกษ์, สภาพแวดล้อมสุดขั้ว
Gravitational Wave	c	สูงมาก	ระยะไกลมาก, สำรอง
DTN Store	N/A	N/A	ขาดการเชื่อมต่อ, หน่วงเวลาสูง

ความเสี่ยง:

ความซับซ้อนในการจำลองช่องทาง 4 แบบ

ประสิทธิภาพเมื่อจำลองหลายโหนด

2.2.2 เลเยอร์ที่ 2: Data Link Layer - Causal-Anchor Addressing Protocol (CAAP)

สถานะการตรวจสอบ: อนุมัติ

รูปแบบแอดเดรส (24 ไบต์):

Causal-Anchor Address (24 bytes):

ฟิลด์	ขนาด	คำอธิบาย
Causal Hash	8B	hash ของลำดับเหตุการณ์
Temporal Index	4B	ดัชนีเวลา (เชิงตรรกะ)
Spatial X	2B	พิกัด X (สัมพัทธ์)
Spatial Y	2B	พิกัด Y (สัมพัทธ์)
Spatial Z	2B	พิกัด Z (สัมพัทธ์)
Stability Score	2B	คะแนนความน่าเชื่อถือ (0-65535)
Metadata	4B	flags, version, channel_pref

การทำงาน:

python

```
class CAAP:
    def __init__(self):
        self.address_cache = {}
        self.resolution_history = []
    def create_address(self, causal_chain, location, stability):
        สร้าง Causal-Anchor Address จากข้อมูล
    def resolve_by_causal_hash(self, causal_hash, temporal_context):
        ค้นหาโหนดจาก Causal Hash
        - ไม่ใช่การค้นหาด้วยตำแหน่ง
        - ค้นหาด้วยประวัติเหตุการณ์
    def calculate_causal_distance(self, ca1, ca2):
        คำนวณระยะห่างเชิงเหตุผล
        - พิจารณา Causal Hash similarity
        - Temporal index difference
        - ไม่ใช่ระยะทางทางกายภาพ
    def select_best_channel(self, source_ca, dest_ca, environment):
        เลือกช่องทางสื่อสารที่เหมาะสมที่สุด
        - ใช้ Neutrino ถ้าต้องทะลุสสาร
        - ใช้ Photon ถ้าต้องการแบนด์วิธสูง
        - ใช้ Gravitational Wave ถ้าระยะไกลมาก
        - ใช้ DTN ถ้าขาดการเชื่อมต่อ
```

ข้อดีของรูปแบบแอดเดรส:

1. Causal Hash เป็นตัวหลัก: ไม่ขึ้นกับตำแหน่ง
2. Temporal Index: รักษาลำดับเหตุการณ์
3. Stability Score: บอกความน่าเชื่อถือ
4. Metadata: รองรับการขยายในอนาคต

2.2.3 เลเยอร์ที่ 3: Network Layer - Temporal Routing Protocol (TRP)

สถานะการตรวจสอบ: อนุมัติแบบมีเงื่อนไข

การออกแบบอัลกอริทึมการค้นหาเส้นทาง:

python

```
class TRP:
    def __init__(self):
        self.routing_table = {}  # {destination: [next_hops]}
        self.causal_topology = {}  # กราฟความสัมพันธ์เชิงเหตุผล
        self.route_cache = {}
    def calculate_route(self, source_ca, dest_ca, constraints):
        ค้นหาเส้นทางด้วย Causal-Aware Dijkstra:
        Cost function:
        C_total = w1 * C_causal + w2 * C_temporal + w3 * C_modal + w4 * C_stability
        โดยที่:
        - C_causal = 1 - causal_similarity(source, dest)
        - C_temporal = |t_source - t_dest| / max_temporal_distance
        - C_modal = channel_adaptation_cost(source, dest, environment)
        - C_stability = 1 - min(stability along path)

        - w1, w2, w3, w4 = น้ำหนัก (configurable)
    def discover_path_by_causality(self, causal_pattern, max_hops):
        ค้นหาเส้นทางตามรูปแบบเหตุ-ผล
        - ไม่ต้องรู้ตำแหน่งปลายทาง
        - ตามหาโหนดที่มี Causal Hash ตรงตาม pattern
    def multi_modal_route_selection(self, source, dest, requirements):
        เลือกเส้นทางตามความต้องการ:
        - high_bandwidth: ใช้ Photon channel
        - high_penetration: ใช้ Neutrino channel
        - extreme_distance: ใช้ Gravitational Wave
        - intermittent: ใช้ DTN store-and-forward
    def update_causal_topology(self, causal_update):
        """อัปเดตกราฟความสัมพันธ์เชิงเหตุผล"""
```

ข้อกำหนดน้ำหนัก (ต้องกำหนดให้ชัดเจน):

น้ำหนัก	ค่าเริ่มต้น	คำอธิบาย
w1 (causal)	0.4	ความคล้ายคลึงเชิงเหตุผลสำคัญที่สุด
w2 (temporal)	0.2	ความแตกต่างเชิงเวลา
w3 (modal)	0.2	ความเหมาะสมของช่องทาง
w4 (stability)	0.2	ความน่าเชื่อถือของโหนด

เงื่อนไขการอนุมัติ:

ต้องกำหนดค่า w1-w4 ให้ชัดเจน

ต้องระบุช่วงเวลาอัปเดต routing table (30 วินาทีในการจำลอง)

เพิ่มกลไก route caching

กำหนดการ converge ของ routing algorithm

2.2.4 เลเยอร์ที่ 4: Transport Layer - Paradox Prevention Protocol (PPP)

สถานะการตรวจสอบ: อนุมัติ

กฎการตรวจจับ Paradox:

รหัสกฎ	คำอธิบาย	การกระทำ
R001	Causal Loop Detected	ป้องกัน loop อนันต์, block หลัง 10 hops
R002	Timeline Inconsistency	ข้อมูลขัดแย้งกับประวัติ, block
R003	Grandfather Paradox	ส่งผลต่อเหตุการณ์ต้นทาง, block
R004	Stability Threshold	คะแนน stability ต่ำเกินไป, block
R005	Channel Mismatch	เลือกช่องทางไม่เหมาะสม, warning

การทำงาน:

python

```
class ParadoxFilter:
    def __init__(self):
```

```
        self.rules_engine = RulesEngine()
        self.causal_history = {}  # {causal_hash: [events]}
        self.violation_log = []
        self.stability_threshold = 0.3  # ค่าต่ำสุดที่ยอมรับ
def validate_packet(self, packet, network_state):
    ตรวจสอบแพ็กเกตตามกฎ Paradox:
        Returns:
        - "ACCEPT": ผ่าน, ส่งต่อได้
        - "BLOCK": ต้อง block
        - "DEFER": รอตรวจสอบเพิ่ม
        - "WARN": ผ่านแต่มีคำเตือน
def check_causal_consistency(self, packet_causal_hash, node_history):
    ตรวจสอบความสอดคล้องเชิงเหตุผล
        - ใช้ Vector Clocks เปรียบเทียบ
        - ต้องไม่ขัดแย้งกับประวัติ
def calculate_paradox_probability(self, packet, path):
    คำนวณความน่าจะเป็นที่จะเกิด Paradox
    ใช้:
        - Causal complexity
        - Temporal distance
        - Node stability scores
def recommend_channel_by_environment(self, source, dest, environment):
    แนะนำช่องทางที่เหมาะสมตามสภาพแวดล้อม
```

การเปรียบเทียบ Paradox Prevention กับ Network Security:

Network Security	UCCN Paradox Prevention
Firewall	Paradox rule engine
Intrusion Detection	Causal inconsistency detection
Access Control	Timeline permissions
Audit Log	Violation logging
DDoS Protection	Loop prevention
Packet Filtering	Causal hash filtering

2.2.5 เลเยอร์ที่ 5: Session Layer - Timeline Synchronization Protocol (TSP)

สถานะการตรวจสอบ: อนุมัติ

การจัดการเซสชัน:

python

```
class TimelineSession:
    def __init__(self, session_id, source_ca, dest_ca):
        self.session_id = session_id
        self.source = source_ca
        self.dest = dest_ca
        self.state = "ESTABLISHING"
        self.vector_clock = {}
        self.channel_preference = None

    def establish_session(self, requirements):
        สร้างเซสชันตามความต้องการ:
        - real_time: ต้องการความหน่วงต่ำ
        - bulk_transfer: ต้องการแบนด์วิธสูง
        - archival: ต้องการความทนทาน
        - exploration: ค้นหาโหนดตาม causal pattern
```

```
def synchronize_causal_clocks(self):
        ปรับ Vector Clocks ให้สอดคล้อง
        - ใช้สำหรับรักษาลำดับเหตุการณ์
def select_session_channel(self, environment):
    เลือกช่องทางสำหรับเซสชัน
        - พิจารณาจากระยะเวลาเซสชัน
        - ปรับเปลี่ยนได้ตามสภาพแวดล้อม
def heartbeat(self):
        """รักษาเซสชันในสภาพขาดการเชื่อมต่อ"""
def migrate_session(self, new_dest_ca):
        """ย้ายเซสชันไปยังโหนดใหม่ (ถ้าโหนดเดิมเสถียรภาพลดลง)"""
```

สถานะเซสชัน:

```
CLOSED ──→ ESTABLISHING ──→ SYNCHRONIZED ──→ ACTIVE ──→ MIGRATING ──→ CLOSED
                                              ↳ SUSPENDED (ขาดการเชื่อมต่อ) ──┘
                                              ↳ RESUMING ──────────────────────┘
```

2.2.6 เลเยอร์ที่ 6: Presentation Layer - Causal Serialization

สถานะการตรวจสอบ: อนุมัติ

การเข้ารหัสและแปลงข้อมูล:

python

```
class CausalSerializer:
    def __init__(self):
        self.encoding_registry = {
        'photon': PhotonEncoder(),
        'neutrino': NeutrinoEncoder(),
        'gravitational': GWEncoder(),
        'dtn': DTNEncoder()
        }

    def encode_for_channel(self, data, channel_type, source_timeline):
        """
      เข้ารหัสข้อมูลให้เหมาะกับช่องทาง:
        - Photon: ใช้ความถี่/เฟส/โพลาไรซ์
        - Neutrino: ใช้ flavor oscillation
        - Gravitational Wave: ใช้ waveform modulation
        - DTN: ใช้ bundle protocol
        """

    def decode_from_channel(self, encoded_data, channel_type, target_timeline):
        """ถอดรหัสข้อมูลจากช่องทาง"""

    def adapt_to_timeline(self, data, source_temporal, target_temporal):
        """
      ปรับข้อมูลให้เข้ากับ timeline ปลายทาง
        - จัดการ temporal drift
        - ปรับ context ให้เหมาะสม
        """
```

2.2.7 เลเยอร์ที่ 7: Application Layer - แอปพลิเคชันสาธิต

สถานะการตรวจสอบ: อนุมัติ

ชุดแอปพลิเคชันสาธิต:

แอปพลิเคชัน	ฟังก์ชัน	การสาธิต
Cosmic Knowledge Sync	ซิงค์ความรู้ข้ามดวงดาว	ส่งข้อความระหว่างโหนดจำลอง
Interstellar Messenger	สื่อสารแบบ real-time	สาธิต latency ต่างๆ
Civilization Archive	เก็บข้อมูลระยะยาว	ทดสอบ DTN store-and-forward
Event Horizon Monitor	เตือนภัยจักรวาล	ทดสอบ multi-channel routing

2.3 ข้อตกลงระหว่างเลเยอร์ (Interface Contracts)

เลเยอร์ N ⟶ เลเยอร์ N-1:

request(service_type, parameters)

เลเยอร์ N-1 ⟶ เลเยอร์ N:

confirm(status, result)

indicate(event, data)

ตัวอย่าง: Network Layer ⟶ Data Link Layer:

python

```
# Network layer ขอ resolve Causal Hash
data_link.request("RESOLVE_CAUSAL", {
    "causal_hash": "0x7A4F...",
    "temporal_context": current_time,
    "required_stability": 0.5
})
# Data link ตอบกลับ
data_link.confirm("SUCCESS", {
    "ca_address": ca_address_object,
    "matching_nodes": [node1, node2],
    "resolution_time": 0.2
})
```

ตัวอย่าง: Data Link Layer ⟶ Physical Layer:

python

```
# Data layer ขอส่งผ่านช่องทางที่เหมาะสม
physical.request("TRANSMIT_MULTI_MODAL", {
    "packet": packet,
    "source_ca": source,
    "dest_ca": dest,
    "environment": {
        "density": "high",  # สสารหนาแน่น ⟶ ใช้ Neutrino
        "distance": 4.37,   # ปีแสง ⟶ พิจารณา GW
        "connectivity": "intermittent"  # ขาดการเชื่อมต่อ ⟶ ใช้ DTN
    }
})
```

2.4 การทบทวนความต้องการที่ไม่ใช่ฟังก์ชัน (Non-Functional Requirements)

ความต้องการ	เป้าหมาย	ปัจจุบัน	สถานะ
Scalability	รองรับ 20+ โหนด	10 โหนด	ต้องพัฒนา
Performance	ตัดสินใจเส้นทาง < 1 วินาที	< 0.5 วินาที	ดี
Reliability	ส่งข้อมูลสำเร็จ 95%	จำลอง	ต้องทดสอบ
Maintainability	โค้ด modular	ดี	ดี
Testability	Coverage >80%	วางแผน	กำลังดำเนินการ
Multi-Modal	รองรับ 4 ช่องทาง	ครบ	ครบ

2.5 สถาปัตยกรรมความปลอดภัย (Paradox Prevention)

โซนความปลอดภัย:

```
┌──────────────────────────────────────────┐
│ โซนที่ 1: Trusted Timeline               │
│ - timeline ปัจจุบัน (ผู้ใช้)             │
│ - Causal hash ที่เชื่อถือได้             │
│ - Stability score > 0.7                  │
├──────────────────────────────────────────┤
```

```
│ โซนที่ 2: Causal DMZ                    │
│ - ตรวจสอบแพ็กเกต                        │
│ - Paradox filtering                     │
│ - Channel adaptation                    │
├─────────────────────────────────────────┤
│ โซนที่ 3: Untrusted Timelines           │
│ - timeline อื่น                         │
│ - Causal hash ที่ไม่คุ้นเคย             │
│ - ต้องตรวจสอบเข้มงวด                    │
└─────────────────────────────────────────┘
```

ส่วนที่ 3: บันทึกการตัดสินใจทางสถาปัตยกรรม

การตัดสินใจที่ 1: Causal Hash เป็นตัวหลักในการอ้างอิง

การตัดสินใจ: ใช้ Causal Hash (8 ไบต์) เป็น primary identifier แทนตำแหน่ง

เหตุผล: ทำให้เครือข่ายไม่ขึ้นกับตำแหน่งทางกายภาพ รองรับการสื่อสารข้ามจักรวาล

ทางเลือกที่พิจารณา: ใช้ตำแหน่ง + เวลา (location-based) -> ไม่เหมาะสมเพราะโหนดเคลื่อนที่

สถานะ: อนุมัติ

การตัดสินใจที่ 2: Multi-Modal Physical Layer

การตัดสินใจ: รองรับ 4 ช่องทางสื่อสาร (Photon, Neutrino, Gravitational Wave, DTN)

เหตุผล: สภาพแวดล้อมจักรวาลหลากหลาย ต้องการช่องทางที่เหมาะสมต่างกัน

ทางเลือกที่พิจารณา: ใช้ช่องทางเดียว (เช่น Photon) -> ไม่เหมาะกับทุกสภาพแวดล้อม

สถานะ: อนุมัติ

การตัดสินใจที่ 3: Causal-Aware Routing

การตัดสินใจ: ใช้ Cost function ที่มี 4 น้ำหนัก (causal, temporal, modal, stability)

เหตุผล: ต้องสมดุลหลายปัจจัยในการสื่อสารข้ามจักรวาล

ทางเลือกที่พิจารณา: ใช้ shortest path แบบเดิม -> ไม่คำนึงถึง causal consistency

สถานะ: ต้องกำหนดน้ำหนักให้ชัดเจน

ส่วนที่ 4: การลงนามรับรองสถาปัตยกรรม

บทบาท	ชื่อ	ลายเซ็น	วันที่	หมายเหตุ
สถาปนิกระบบ	นายวงศธร ธน.ยอด	วงศธร	22/02/2026	
วิศวกรเครือข่าย	นายพัชรพล กองแก้ว	พัชรพล	22/02/2026	
ผู้เชี่ยวชาญเฉพาะด้าน	นายกฤษฎา นามมนต์เทียน	กฤษฎา	22/02/2026	
ผู้ดูแลระบบและเครื่องมือ	นายศุภกิตติ์ ฟันเฟือย	ศุภกิตติ์	22/02/2026	
ผู้ทดสอบ/ประกันคุณภาพ	นายคมชาญ น้อยเนียม	คมชาญ	22/02/2026	

ผลการตรวจสอบ: อนุมัติแบบมีเงื่อนไข

เงื่อนไขที่ต้องทำให้เสร็จภายใน: สัปดาห์ที่ 2 วันที 3

กำหนดค่า w1, w2, w3, w4 ใน TRP ให้ชัดเจน

ระบุช่วงเวลาอัปเดต routing table

เพิ่มกลไกคำนวณ Paradox probability

ภาคผนวก

ภาคผนวก A: อภิธานศัพท์

ศัพท์	คำจำกัดความ
Causal Hash	ค่าแฮชที่เกิดจากลำดับเหตุการณ์ ใช้ระบุตัวตน
Temporal Index	ดัชนีเชิงเวลาแบบตรรกะ (Logical Time)
Stability Score	คะแนนความน่าเชื่อถือของโหนด (0-1)
Paradox	เหตุการณ์ที่ขัดแย้งกับความสัมพันธ์เชิงเหตุผล
Multi-Modal	การรองรับช่องทางสื่อสารหลายแบบ

ภาคผนวก B: เอกสารอ้างอิง

Kurose & Ross, "Computer Networking: A Top-Down Approach"

Tanenbaum, "Computer Networks"

Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System"

Pearl, "Causality: Models, Reasoning, and Inference"

RFC 4838 - Delay-Tolerant Networking Architecture

ไฟล์ที่ 2: implementation_plan.md

Universal Causality-based Communication Network (UCCN) - แผนการดำเนินงานเวอร์ชัน 1.0

การควบคุมเอกสาร

เวอร์ชัน	วันที่	ผู้จัดทำ	บทบาท	การเปลี่ยนแปลง
v1.0	22/02/2026	นายศุภกิตติ์ ฟันเฟือย	ผู้ดูแลระบบ	จัดทำแผนการดำเนินงานเบื้องต้น

การมอบหมายบทบาททีม

บทบาท	ผู้รับผิดชอบ	ความรับผิดชอบหลัก	ความรับผิดชอบรอง
สถาปนิกระบบ	นายวงศธร ธน.ยอด	ออกแบบระบบ, กำหนด interface, ตัดสินใจสถาปัตยกรรม	ตรวจสอบโค้ด, ดูแลเอกสาร
วิศวกรเครือข่าย	นายพัชรพล กองแก้ว	พัฒนาโพรโทคอล, อัลกอริทึมหลัก, จำลองสถานการณ์	สนับสนุนการทดสอบ, แก้ไขบั๊ก
ผู้เชี่ยวชาญเฉพาะด้าน	นายกฤษฎา นามมนต์เทียน	กฎ Paradox, ความสัมพันธ์เชิงเหตุผล, องค์ความรู้เฉพาะทาง	ดูแลเอกสาร, ตรวจสอบความถูกต้อง
ผู้ดูแลระบบและเครื่องมือ	นายศุภกิตติ์ ฟันเฟือย	จัดการสภาพแวดล้อม, CI/CD, บูรณาการ, ดูแล repository	ดูแลเอกสาร, การ deploy
ผู้ทดสอบ/ประกันคุณภาพ	นายคมชาญ น้อยเนียม	แผนทดสอบ, กรณีทดสอบ, ตัวชี้วัดคุณภาพ, ตรวจสอบ	ติดตามบั๊ก, รายงานผล

ส่วนที่ 1: การวิเคราะห์การดำเนินงาน

1.1 การประเมินความซับซ้อน

ส่วนประกอบ	ความซับซ้อน (1-5)	ระดับความเสี่ยง	เวลาที่ประมาณการ (ชั่วโมง)
CAAP (Addressing)	3	ต่ำ	8-10
TRP (Routing)	5	สูง	20-25

PPP (Paradox)	4	ปานกลาง	12-15
TSP (Session)	3	ต่ำ	8-10
Multi-Modal Physical	4	ปานกลาง	15-18
Simulation Framework	4	ปานกลาง	15-20
Visualization	3	ต่ำ	6-8
Testing Suite	3	ปานกลาง	10-12
Documentation	2	ต่ำ	ตลอดโครงการ

รวมเวลาที่ประมาณการ: 94-118 ชั่วโมง

เวลาที่ทีมมีทั้งหมด (4 สัปดาห์ × 5 คน × 6 ชั่วโมง/สัปดาห์): 120 ชั่วโมง

เวลาสำรอง: 2-26 ชั่วโมง (2-22%)

1.2 การวิเคราะห์การขึ้นต่อกัน

```
สัปดาห์ 1                          สัปดาห์ 2              สัปดาห์ 3          สัปดาห์ 4

┌──────────────────┐   ┌──────────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│ ออกแบบ           │──>│ พัฒนา                │─>│ บูรณาการ         │>│ ส่งมอบ           │
│ สถาปัตยกรรม      │   │ โพรโทคอล             │   │ และนำเสนอ        │   │                  │
└──────────────────┘   │ และทดสอบ             │   └──────────────────┘   └──────────────────┘
                       └──────────────────────┘

        ↓                      ↓                      ↓                      ↓
CAAP Spec ──────────> CAAP Code ──────────> CAAP Test ──────────> CAAP Integrated
TRP Spec ──────────> TRP Code ──────────> TRP Test ──────────> TRP Integrated
PPP Spec ──────────> PPP Code ──────────> PPP Test ──────────> PPP Integrated
TSP Spec ──────────> TSP Code ──────────> TSP Test ──────────> TSP Integrated
Multi-Modal Spec ─> Multi-Modal Code ─> Multi-Modal Test ─> Multi-Modal Integrated
Sim Setup ──────────> Sim Dev ──────────> Sim Test ──────────> Sim Final
```

Critical Path: TRP Implementation ⟶ Integration ⟶ Testing

งานที่ทำคู่ขนานได้: CAAP, PPP, TSP, Multi-Modal, Visualization, Documentation

ส่วนที่ 2: แผนการดำเนินงาน 4 สัปดาห์

สัปดาห์ที่ 1: วางรากฐาน (วันที่ 1-5)

ธีม: ออกแบบสถาปัตยกรรม, ตั้งค่าสภาพแวดล้อม, ออกแบบส่วนประกอบ

วันที่ 1: เริ่มโครงการและตั้งค่าสภาพแวดล้อม

เวลา	**กิจกรรม**	**ผู้นำ**	**ผู้เข้าร่วม**
9:00-10:00	วางแผน Sprint	DevOps	ทุกคน

10:00-12:00	ตั้งค่าสภาพแวดล้อม	DevOps	ทุกคน
13:00-15:00	ทบทวนสถาปัตยกรรม	Architect	ทุกคน
15:00-17:00	งานตามบทบาท	แต่ละคน	รายบุคคล

สิ่งที่ต้องส่งมอบ:

สร้าง GitHub repository (DevOps)

เอกสารการตั้งค่าสภาพแวดล้อม (DevOps)

ติดตั้ง requirements (ทุกคน)

แผนภาพสถาปัตยกรรมเบื้องต้น (Architect)

งานตามบทบาท:

บทบาท	**งาน**
Architect	กำหนด interface ระหว่างเลเยอร์, สร้าง interface contracts
Engineer	ศึกษา Python/NetworkX, สร้าง prototype กราฟอย่างง่าย
Specialist	วิจัยกฎ Paradox, ร่างกฎ 5 ข้อ
DevOps	ตั้งค่า CI (GitHub Actions), สร้าง project board
Tester/QA	สร้าง template แผนทดสอบ, กำหนดประเภทการทดสอบ

วันที่ 2: ออกแบบ CAAP และ Physical Layer

Focus: CAAP specification และ Multi-Modal simulation

Pair Programming:

- Architect + Engineer: ออกแบบรูปแบบ CAAP
- Specialist + Tester: เอกสารกฎ Paradox

สิ่งที่ต้องส่งมอบ:

- CAAP address format specification (Architect)
- Causal Hash algorithm (Engineer)
- Multi-Modal channel prototype (Engineer)
- กฎ Paradox v0.1 (Specialist)
- กรณีทดสอบสำหรับ CAAP (Tester)

วันที่ 3: ออกแบบ TRP (Routing)

Focus: TRP algorithm design

สิ่งที่ต้องส่งมอบ:

- TRP routing algorithm pseudo-code (Engineer)

- Cost function definition (Architect + Specialist)
- Routing table structure (Engineer)
- กรณีทดสอบสำหรับ TRP (Tester)

การตัดสินใจสำคัญ: กำหนดค่า w1, w2, w3, w4

วันที่ 4: ออกแบบ PPP และ TSP

Focus: Paradox Prevention และ Session Management

สิ่งที่ต้องส่งมอบ:

- กฎ Paradox ฉบับสมบูรณ์ (Specialist)
- Rule engine design (Engineer)
- Session state machine (Architect)
- กรณีทดสอบสำหรับ PPP (Tester)

วันที่ 5: ทบทวนสัปดาห์ที่ 1 และวางแผนสัปดาห์ที่ 2

กิจกรรม:

- 14:00-15:00: ตรวจสอบโค้ด
- 15:00-16:00: ทดสอบการบูรณาการ
- 16:00-17:00: ทบทวน Sprint และวางแผนสัปดาห์ที่ 2

เกณฑ์ความสำเร็จสัปดาห์ที่ 1:

- เอกสาร specification ครบทุกส่วน
- สภาพแวดล้อมพัฒนาทำงานได้ทุกคน
- สร้าง simulation framework พื้นฐาน
- ตั้ง test framework
- CI ผ่านบน main branch

สัปดาห์ที่ 2: พัฒนาโพรโทคอลหลัก (วันที่ 6-10)

ธีม: Implement โพรโทคอลหลัก

วันที่ 6-7: Implement CAAP และ Physical Layer

Lead: Engineer

Support: Architect (code review), Tester (test cases)

งาน:

1. Implement CausalAnchorAddress class
2. สร้าง Causal Hash module
3. สร้าง Multi-Modal channel simulation

4. เขียน unit tests (minimum 80% coverage)

Definition of Done:

- สร้างและ validate address ได้
- resolve โดย Causal Hash ได้
- channel simulation ทำงาน
- tests ผ่านทั้งหมด

วันที่ 8-9: Implement TRP (Routing)

Lead: Engineer

Support: Specialist (routing metrics), Tester (test scenarios)

งาน:

1. Implement routing table management
2. สร้าง Causal-Aware Dijkstra
3. สร้าง route calculation module
4. เพิ่ม route caching

Critical Path Item: ต้องเสร็จภายในสิ้นวันที่ 9

วันที่ 10: Implement PPP และ TSP

Lead: Specialist + Engineer

Support: Tester (validation)

งาน:

1. Implement paradox rule engine
2. สร้าง packet validation pipeline
3. Implement session management
4. ทดสอบกับ paradox scenarios

เกณฑ์ความสำเร็จสัปดาห์ที่ 2:

- โพรโทคอลหลักทั้งหมด implement
- Unit tests ผ่าน (>80% coverage)
- แต่ละส่วนทำงานได้อิสระ
- อัปเดตเอกสาร

สัปดาห์ที่ 3: บูรณาการและทดสอบ (วันที่ 11-15)

ธีม: รวมส่วนประกอบ ทดสอบ และปรับปรุง

วันที่ 11: บูรณาการส่วนประกอบ - เฟส 1

ลำดับการบูรณาการ:

1. CAAP + Physical Layer
2. เพิ่ม TRP
3. เพิ่ม PPP

Lead การบูรณาการ: DevOps

Lead การทดสอบ: Tester

กิจกรรม:

- เช้า: บูรณาการ CAAP + Physical
- บ่าย: เพิ่ม TRP, ทดสอบ routing
- เย็น: รัน integration tests

วันที่ 12: บูรณาการส่วนประกอบ - เฟส 2

งานบูรณาการ:

1. เพิ่ม PPP เข้าใน stack
2. เพิ่ม TSP
3. สร้าง end-to-end packet flow

Milestone: ส่ง packet end-to-end สำเร็จครั้งแรกภายในสิ้นวัน

วันที่ 13: ทดสอบระบบและแก้ไขบั๊ก

กิจกรรมการทดสอบ:

ชุดทดสอบ	**เจ้าของ**	**เป้าหมาย**
Unit Tests	Engineer	รันซ้ำทั้งหมด
Integration Tests	Tester	20 scenarios
Performance Tests	DevOps	response time <1s
Paradox Scenarios	Specialist	10 edge cases
Multi-Modal Tests	Tester	ทดสอบการสลับช่องทาง

วันที่ 14: พัฒนา Visualization และ Demo

Lead: Engineer

Support: ทุกคน

ความต้องการ Visualization:

- แสดง topology (Causal Graph)
- แสดงการเลือกช่องทาง
- แสดงการตรวจจับ Paradox
- แสดง Causal Hash และ Temporal Index

วันที่ 15: ทบทวนสัปดาห์ที่ 3 และทดสอบระบบ

กิจกรรม:

- 10:00-12:00: ทดสอบระบบเต็มรูปแบบ
- 13:00-15:00: แก้ไขบั๊ก
- 15:00-17:00: ซ้อมนำเสนอภายในทีม

เกณฑ์ความสำเร็จสัปดาห์ที่ 3:

- ส่วนประกอบทั้งหมดรวมกันได้
- ส่ง packet end-to-end ได้
- 3 demo scenarios ทำงานได้
- Visualization แสดงถูกต้อง
- Test coverage >80%

สัปดาห์ที่ 4: ส่งมอบและนำเสนอ (วันที่ 16-20)
ธีม: ทำให้เสร็จสมบูรณ์ เอกสาร และนำเสนอ
วันที่ 16: เขียนเอกสาร

เอกสาร	**เจ้าของ**	**ที่อยู่**
User Guide	DevOps	docs/user_guide.md
API Reference	Engineer	docs/api.md
Test Report	Tester	docs/test_report.md
Architecture Final	Architect	architecture_spec.md
Implementation Summary	ทุกคน	README.md

วันที่ 17: ปรับแต่งและเพิ่มประสิทธิภาพ
จุดเน้น:

- ทำความสะอาดโค้ดและเพิ่ม comments
- เพิ่มประสิทธิภาพ (optimization)
- จัดการ edge cases
- ปรับปรุง visualization

วันที่ 18: เตรียมการนำเสนอ

งาน:

1. สร้างสไลด์นำเสนอ (ทุกคน)
2. บันทึกวิดีโอสาธิต (Engineer)
3. เตรียมสคริปต์สาธิตสด (Tester)
4. ซ้อมนำเสนอ (ทุกคน)

โครงสร้างการนำเสนอ (15 นาที):

ส่วน	**ระยะเวลา**	**ผู้นำเสนอ**
บทนำ	2 นาที	Architect
สถาปัตยกรรม	3 นาที	Architect
การพัฒนา	3 นาที	Engineer
Paradox Prevention	2 นาที	Specialist
สาธิต	3 นาที	Engineer
ผลการทดสอบ	1 นาที	Tester
สรุป	1 นาที	DevOps

วันที่ 19: ตรวจสอบครั้งสุดท้ายและซ้อม

กำหนดการ:

- 9:00-11:00: ตรวจสอบโค้ดครั้งสุดท้าย
- 11:00-12:00: ตรวจสอบเอกสาร
- 13:00-15:00: ซ้อมนำเสนอ
- 15:00-16:00: รับฟัง feedback และแก้ไข
- 16:00-17:00: ปรับครั้งสุดท้าย

วันที่ 20: วันส่งงาน

Checklist สุดท้าย:

โค้ด:

- โค้ดทั้งหมด push ไปที่ main branch
- tag เป็น v1.0.0
- README อัปเดตพร้อมวิธีติดตั้ง
- requirements.txt ครบถ้วน

เอกสาร:

- architecture_spec.md
- implementation_plan.md
- user_guide.md
- api.md
- test_report.md

การนำเสนอ:

- สไลด์นำเสนอ (PDF)
- วิดีโอสาธิต (MP4)
- พร้อมสาธิตสด

สิ่งที่ส่งมอบ:

- ลิงก์ GitHub repository
- ไฟล์ zip รวม deliverables
- รายงานการมีส่วนร่วมของทีม
- การประเมินเพื่อนร่วมทีม

ส่วนที่ 3: การวิเคราะห์การดำเนินงานตามบทบาท

3.1 การวิเคราะห์ของสถาปนิกระบบ (Architect)

ข้อกังวลหลัก:

- ความเสถียรของ interface ระหว่างเลเยอร์
- ความเข้ากันได้ของโพรโทคอล
- ความสามารถในการขยายของระบบแอดเดรส

Checklist การดำเนินงาน:

- interface ทุกเลเยอร์ documented
- CAAP format ทดสอบกับกรณีต่างๆ
- รูปแบบโพรโทคอล specified
- บันทึกการตัดสินใจทางสถาปัตยกรรม

การลดความเสี่ยง:

- สร้าง interface mocks สำหรับการพัฒนาคู่ขนาน
- ตรวจสอบโค้ดทุกครั้งกับ architecture spec
- ติดตาม architectural drift

3.2 การวิเคราะห์ของวิศวกรเครือข่าย (Engineer)

งานหลักตามลำดับความสำคัญ:

Priority 1 (Must Have):

- CausalAnchorAddress class
- Causal Hash generation
- RoutingTable management
- Dijkstra with causal weights
- Packet structure
- Multi-Modal channel simulation
- Basic simulation

Priority 2 (Should Have):

- Route caching
- Multi-path routing
- Visualization
- Performance optimization

Priority 3 (Nice to Have):

- Load balancing
- Advanced metrics
- GUI interface
- Real-time updates

ความท้าทายทางเทคนิค:

- การค้นหาเส้นทางที่มี causal weights
- ประสิทธิภาพเมื่อมี 20+ โหนด
- การบูรณาการกับ visualization

3.3 การวิเคราะห์ของผู้เชี่ยวชาญเฉพาะด้าน (Specialist)

การ implement กฎเฉพาะทาง:

กฎ	**ความซับซ้อน**	**วิธีการตรวจสอบ**
Causal Loop Detection	ปานกลาง	จำกัด hop count, ติดตาม path
Timeline Inconsistency	สูง	ใช้ Vector Clocks เปรียบเทียบ
Grandfather Paradox	สูง	วิเคราะห์ causal dependency
Stability Threshold	ต่ำ	ตรวจสอบคะแนน stability
Channel Recommendation	ปานกลาง	พิจารณาสภาพแวดล้อม

ความต้องการวิจัย:

- ทบทวน Causal Theory
- เอกสาร 5 paradox scenarios
- สร้าง paradox probability model
- กำหนด edge cases

3.4 การวิเคราะห์ของผู้ดูแลระบบ (DevOps)

ความต้องการโครงสร้างพื้นฐาน:

ส่วนประกอบ	**เทคโนโลยี**	**การตั้งค่า**
Version Control	GitHub	Branch protection
CI/CD	GitHub Actions	Python 3.8+
Documentation	MkDocs	Material theme
Testing	pytest	Coverage reporting
Dependencies	pip	requirements.txt

CI Pipeline:

```
name: UCCN CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Setup Python
        uses: actions/setup-python@v2
        with:
          python-version: '3.8'
```

```
    - name: Install dependencies
      run: |
        pip install -r requirements.txt
        pip install pytest pytest-cov flake8
    - name: Run tests
      run: pytest tests/ --cov=src --cov-report=xml
    - name: Check style
      run: flake8 src/
    - name: Upload coverage
      uses: codecov/codecov-action@v1
```

3.5 การวิเคราะห์ของผู้ทดสอบ (Tester/QA)

กลยุทธ์การทดสอบ:

ระดับการทดสอบ:

1. Unit Tests (สัปดาห์ที่ 2)
    - ทดสอบทีละ class
    - method-level validation
    - คาดหวัง: 60+ test cases
2. Integration Tests (สัปดาห์ที่ 3)
    - ทดสอบ interaction ระหว่างเลเยอร์
    - End-to-end packet flow
    - คาดหวัง: 25+ scenarios
3. System Tests (สัปดาห์ที่ 4)
    - ทดสอบ simulation เต็มรูปแบบ
    - Performance benchmarks
    - คาดหวัง: 15+ test runs

Matrix การทดสอบ:

ส่วนประกอบ	Unit Tests	Integration	System	เจ้าของ
CAAP	15	5	2	Tester
TRP	20	8	3	Tester
PPP	15	5	3	Tester
TSP	10	4	2	Tester
Multi-Modal	10	3	2	Tester
Simulation	10	2	3	Tester

Template การรายงานบั๊ก:

รายงานบั๊ก

ID: UCCN-[หมายเลข]

ความรุนแรง: Critical/High/Medium/Low

ส่วนประกอบ: [CAAP/TRP/PPP/TSP/Multi-Modal/Sim]

คำอธิบาย:

ขั้นตอนการทำให้เกิด:

พฤติกรรมที่คาดหวัง:

พฤติกรรมที่เกิดขึ้นจริง:

มอบหมายให้:

สถานะ: Open/In Progress/Resolved/Verified

ส่วนที่ 4: Checklist การทบทวน Sprint

สิ้นสุด Sprint 1 (สัปดาห์ที่ 1)

- สถาปัตยกรรม approved
- Environment ทำงานได้
- Test framework พร้อม
- Specs ครบถ้วน
- วัด team velocity ได้

สิ้นสุด Sprint 2 (สัปดาห์ที่ 2)

- โพรโทคอลหลัก implemented
- Unit tests ผ่าน
- แต่ละ component ทำงานได้
- โค้ดผ่านการ review

สิ้นสุด Sprint 3 (สัปดาห์ที่ 3)

- Components ทั้งหมด integrated
- End-to-end flow ทำงาน
- Demo scenarios พร้อม
- Visualization ทำงาน

สิ้นสุด Sprint 4 (สัปดาห์ที่ 4)

- deliverables ทั้งหมดพร้อม
- Documentation สมบูรณ์
- Presentation พร้อม
- Submission package พร้อม

ส่วนที่ 5: แผนสำรอง

แผน A: ตามหลังกำหนดการ

ถ้าตามหลัง >2 วัน:

- ลด scope (remove nice-to-have features)
- เพิ่ม pair programming
- ขยายเวลาทำงาน (max +2 ชั่วโมง/วัน)

แผน B: Integration ล้มเหลว

ถ้า integration ล้มเหลว:

- roll back ไป version สุดท้ายที่ใช้ได้
- isolate component ที่มีปัญหา
- สร้าง simplified interface
- re-integrate ด้วย mocks

แผน C: สมาชิกทีมขาด

ถ้าสมาชิกขาด >2 วัน:

- แจกจ่ายงาน
- pair programming
- ปรับ deliverables
- document knowledge transfer

แผน D: Technical Block

ถ้าเจอปัญหาทางเทคนิค:

- จำกัดเวลาค้นคว้า 2 ชั่วโมง
- escalate ทีม
- ขอคำแนะนำอาจารย์
- implement workaround